

MODULE 4

UNSUPERVISED MACHINE LEARNING AND ADVANCED TECHNIQUES FOR INSIGHTS

MLFP — ML Foundations for Professionals — Terrene Open Academy

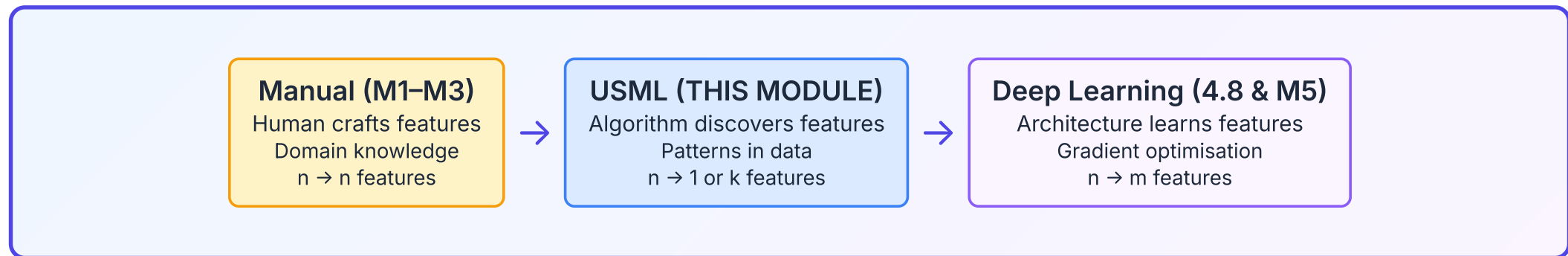
"The algorithm that found \$2 billion in hidden fraud had never seen a single labelled example."

RECAP: YOUR JOURNEY SO FAR

Module	What You Learned	Kailash Engines
M1	Data Pipelines & Visualisation	DataExplorer, PreprocessingPipeline
M2	Statistical Foundations & Experiment Design	ExperimentTracker, ModelVisualizer
M3	Supervised ML — regression, classification, trees, boosting	TrainingPipeline, HyperparameterSearch, ModelRegistry

M1–M3 gave you features engineered by hand and models trained with labels. Today: what happens when nobody tells you the right answer — and the features engineer themselves?

THE FEATURE ENGINEERING SPECTRUM



This is the most important slide in the module. USML is NOT a separate discipline — it is the bridge between hand-crafted features and deep-learned representations.

MODULE 4 ROAD MAP

Lesson	Topic	Spectrum Position
4.1	Clustering	Group discovery ($n \rightarrow 1$)
4.2	EM Algorithm & GMMs	Soft clustering — probabilistic assignment
4.3	PCA, t-SNE, UMAP	Feature compression ($n \rightarrow k$)
4.4	Anomaly Detection	Outlier discovery
4.5	Market Basket Analysis	Co-occurrence patterns
4.6	NLP & Topic Modelling	Text feature discovery
4.7	Recommender Systems	THE PIVOT — optimisation drives features
4.8	DL Foundations	Hidden layers = USML + error feedback

The arc: from clustering (simplest USML) through the pivot at 4.7 (optimisation discovers embeddings) to 4.8 (neural networks generalise all of it).

4.1 CLUSTERING

Discovering group structure without labels

K-MEANS: THE WORKHORSE

1. Choose k centroids randomly
2. Assign each point to the nearest centroid
3. Recompute centroids as cluster means
4. Repeat until assignments stabilise

$$J = \sum_{k=1}^K \sum_{x \in C_k} \|x - \mu_k\|^2$$

K-means objective: minimise within-cluster sum of squared distances

Sensitivity: Random init can converge to local minima. **k-means++** selects initial centroids spread apart.

CHOOSING K: THE ELBOW METHOD

- Plot inertia (J) vs number of clusters k
- Look for the "elbow" where adding clusters gives diminishing returns
- Subjective — complement with silhouette analysis

Gap statistic: compare within-cluster dispersion to a null reference distribution. More principled than visual elbow.

HIERARCHICAL CLUSTERING

AGGLOMERATIVE (BOTTOM-UP)

1. Each point = own cluster
2. Merge two closest clusters
3. Repeat until one cluster remains

LINKAGE METHODS

Method	Distance	Effect
Single	Min pairwise	Chains; elongated clusters
Complete	Max pairwise	Compact, spherical
Average	Mean pairwise	Balanced
Ward's	Variance increase	Min variance; most popular

Dendrogram: tree diagram showing merge history. Cut at a threshold to obtain clusters. No need to pre-specify k.

DBSCAN & HDBSCAN

DBSCAN

- **Core point:** $\geq \text{minPts}$ within ε
- **Border point:** within ε of core
- **Noise point:** neither
- Finds arbitrary-shaped clusters
- Does NOT require k

HDBSCAN

- Hierarchical extension — auto-selects ε
- Handles varying density
- Outputs cluster membership probabilities
- **Best general-purpose density-based method**

SPECTRAL CLUSTERING

- Construct similarity graph from data
- Compute graph Laplacian: $L = D - W$
- Eigendecompose L ; use bottom- k eigenvectors as features
- Apply K-means in this new space

When to use: non-convex clusters where K-means and hierarchical fail (e.g., concentric rings, interleaving spirals).

CLUSTER EVALUATION: INTERNAL METRICS

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Silhouette score: $a(i)$ = avg intra-cluster distance, $b(i)$ = avg nearest-cluster distance. Range: $[-1, 1]$

DAVIES-BOULDIN

$$DB = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \frac{s_i + s_j}{d(c_i, c_j)}$$

Lower = better. Compares cluster spread to inter-cluster distance.

CALINSKI-HARABASZ

$$CH = \frac{SS_B / (k-1)}{SS_W / (n-k)}$$

Higher = better. Ratio of between- to within-cluster variance.

CLUSTER EVALUATION: EXTERNAL METRICS

Metric	What It Measures	Range	When to Use
ARI	Agreement with ground truth (chance-adjusted)	$[-1, 1]$	Have true labels
NMI	Mutual information (normalised)	$[0, 1]$	Comparing clusterings
Gap Statistic	Dispersion vs null reference	N/A	Choosing k

Key insight: Without labels, you cannot compute ARI/NMI. In practice, you often combine internal metrics with domain expert validation.

ALGORITHM SELECTION GUIDE

Algorithm	Best For	Limitations	Needs k?
K-Means	Spherical, equal-size clusters	Sensitive to init & outliers	Yes
Hierarchical	Small data, need dendrogram	$O(n^2)$ memory	No
DBSCAN	Arbitrary shapes, noisy data	Struggles with varying density	No
HDBSCAN	Varying density, production	Slower than K-Means	No
Spectral	Non-convex structures	Expensive for large n	Yes
GMM	Soft assignments, overlapping	Assumes Gaussian shape	Yes

APPLICATION: CUSTOMER SEGMENTATION

SINGAPORE RETAIL CASE

- RFM features: Recency, Frequency, Monetary value
- K-means identifies 4 segments: loyal, at-risk, new, churned
- HDBSCAN discovers a 5th micro-segment: seasonal bulk buyers
- Cluster membership becomes a **feature** for churn prediction (M3 pipeline)

USML produces features: cluster_id is a new column you feed into your supervised model from M3.

KAILASH BRIDGE: AUTOMLENGINE FOR CLUSTERING

THEORY → ENGINE

K-means, GMM, HDBSCAN, Spectral, Hierarchical → AutoMLEngine

```
from kailash_ml import AutoMLEngine, AutoMLConfig

config = AutoMLConfig(
    task="clustering",
    algorithms=["kmeans", "gmm", "hdbscan", "spectral"],
    metric="silhouette",
    n_trials=50
)

engine = AutoMLEngine(config)
results = engine.fit(df)

best = results.best_model
print(f"Best: {best.algorithm}, Silhouette: {best.score:.3f}")
labels = best.labels          # Hard assignments
probabilities = best.probas    # Soft assignments (if available)
```

EXERCISE 4.1: CUSTOMER SEGMENTATION

Task: Segment retail customers using RFM features.

- Apply K-means, hierarchical (with dendrogram), and HDBSCAN
- Evaluate with silhouette score and Davies-Bouldin index
- Interpret clusters with business meaning (not just "cluster 0, cluster 1")
- Use AutoML Engine to compare all algorithms programmatically

4.2 EM ALGORITHM & GAUSSIAN MIXTURE MODELS

Soft clustering — probabilistic assignment to groups

HARD VS SOFT CLUSTERING

K-MEANS (HARD)

- Each point $\rightarrow$ exactly 1 cluster
- Binary assignment: in or out
- Cannot express uncertainty

GMM (SOFT)

- Each point $\rightarrow$ probability per cluster
- $P(\text{cluster}_k \mid x) = 0.6, 0.3, 0.1$
- Expresses uncertainty naturally

Soft assignments give k probability features per point — richer than a single cluster label.

THE EM ALGORITHM: E-STEP

E-STEP: COMPUTE RESPONSIBILITIES

For each point x_n and cluster k , compute the probability that x_n belongs to cluster k :

$$r_{nk} = \frac{\pi_k \mathcal{N}(x_n \mid \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_n \mid \mu_j, \Sigma_j)}$$

π_k = mixing coefficient (prior), $\mathcal{N}$ = Gaussian density

Intuition: "How much does cluster k explain this point?" Responsibilities are normalised — they sum to 1 for each point.

THE EM ALGORITHM: M-STEP

M-STEP: UPDATE PARAMETERS

Using responsibilities as weights, recompute cluster parameters:

$$\mu_k = \frac{\sum_n r_{nk} x_n}{\sum_n r_{nk}} \quad \pi_k = \frac{\sum_n r_{nk}}{N}$$

Weighted mean (soft K-means) and updated mixing coefficients

Covariance update: $\Sigma_k = \frac{\sum_n r_{nk} (x_n - \mu_k)(x_n - \mu_k)^T}{\sum_n r_{nk}}$

EM CONVERGENCE & LOG-LIKELIHOOD

$$\mathcal{L} = \sum_{n=1}^N \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(x_n \mid \mu_k, \Sigma_k) \right)$$

Log-likelihood is guaranteed non-decreasing at every EM iteration

- EM alternates E-step and M-step until $\mathcal{L}$ converges
- Converges to a **local** maximum (not necessarily global)
- Run multiple random initialisations; pick the best $\mathcal{L}$

GMM: EM APPLIED TO GAUSSIANS

- A GMM models data as a mixture of K Gaussian distributions
- Each component has its own μ_k , Σ_k , and π_k
- EM finds the parameters that best explain the data
- Soft assignments = posterior responsibilities r_{nk}

EM as a general template: EM works for ANY latent variable model, not just Gaussians. It is the backbone of LDA (4.6), matrix factorisation (4.7), and the training of neural networks (4.8).

MIXTURE OF EXPERTS: MODERN APPLICATION

From GMM to MoE:

- GMM: each component is a Gaussian → soft assignment
- MoE: each component is a **neural network expert** → gating network selects which expert processes which input
- Modern LLM architectures (e.g., GPT-4 class) use Sparse MoE

Preview (M5+): The gating network is itself a small neural network that learns to route inputs. Same principle as GMM responsibilities — scaled to billions of parameters.

EXERCISE 4.2: EM FROM SCRATCH

Task: Implement the EM algorithm in ~20 lines on 2D synthetic data (3 Gaussians).

- Implement E-step and M-step from the formulas
- Verify: responsibilities sum to 1 for each point
- Compare your implementation with GMM on real e-commerce data
- Visualise soft assignments (colour by max responsibility, opacity by confidence)

4.3 DIMENSIONALITY REDUCTION

Feature compression — discovering latent axes

PCA STEP 1: DECORRELATE

- Rotate axes to align with directions of maximum variance
- New axes = **principal components** (PCs)
- PCs are orthogonal (uncorrelated)

$$\max_{\mathbf{w}} \text{Var}(\mathbf{w}^T \mathbf{X}) \quad \text{subject to} \quad \|\mathbf{w}\| = 1$$

First PC maximises projected variance; subsequent PCs are orthogonal

PCA STEP 2: REDUCE

- Keep top- k components by variance explained
- Drop the rest — they capture noise

$$\text{Variance explained}_k = \frac{\lambda_k}{\sum_i \lambda_i}$$

λ_k = eigenvalue of the k -th component. Scree plot shows this.

Scree plot: plot variance explained vs component number. Choose k where the curve flattens.

PCA VIA SVD

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$$

$\mathbf{U}$ = left singular vectors, $\mathbf{\Sigma}$ = singular values, $\mathbf{V}$ = right singular vectors (loadings)

- Principal components = columns of $\mathbf{U}\mathbf{\Sigma}$
- Loadings = columns of $\mathbf{V}$ — interpret what each PC represents
- SVD is numerically more stable than eigendecomposition

Connection to 4.7: Matrix factorisation for recommenders also uses SVD. PCA factorises $\mathbf{X}$; collaborative filtering factorises the user-item matrix $\mathbf{R}$. Same algebra, different data.

RECONSTRUCTION ERROR

$$\|\mathbf{X} - \hat{\mathbf{X}}\|^2 = \sum_{i=k+1}^p \lambda_i$$

Information lost = sum of discarded eigenvalues

Trade-off: fewer components = more compression but more information loss. Choose k to retain (e.g.) 95% of total variance.

KERNEL PCA

- Standard PCA finds **linear** directions of max variance
- Kernel PCA maps data to higher-dimensional space via kernel trick
- Common kernels: RBF (Gaussian), polynomial
- Captures nonlinear structure that PCA misses

Limitation: No explicit loadings — harder to interpret. No inverse transform. Use when visualisation or downstream accuracy matters more than interpretability.

T-SNE: VISUALISATION SPECIALIST

HOW IT WORKS

- Convert distances to probabilities in high-D
- Find low-D layout with similar probabilities
- Minimise KL divergence between distributions

KEY PROPERTIES

- **Perplexity** (~5-50): effective number of neighbours
- Preserves local structure well
- Distorts global structure
- Non-deterministic
- **NOT for feature extraction** (no inverse transform)

UMAP: THE MODERN DEFAULT

	t-SNE	UMAP
Speed	Slow ($O(n^2)$)	Fast ($O(n \log n)$)
Global structure	Distorted	Better preserved
Deterministic	No	Yes (with seed)
Feature extraction	No inverse	Has inverse — can be used as features
Scalability	~10k points	~1M+ points

Practical rule: Use UMAP for both visualisation AND as features for downstream models. Use t-SNE only when you want a second opinion on local structure.

MANIFOLD LEARNING: REFERENCE TABLE

Method	Idea	Best For
PCA	Linear variance maximisation	General preprocessing, interpretable
Kernel PCA	Nonlinear via kernel trick	Nonlinear structure, medium n
t-SNE	Probability matching (local)	Visualisation only
UMAP	Topological structure	Visualisation + features
Isomap	Geodesic distances	Data on a curved manifold
LLE	Local linear reconstruction	Swiss roll, manifold unfolding
MDS	Preserve pairwise distances	Distance matrices (not feature matrices)

INTRINSIC DIMENSIONALITY

- How many dimensions does the data *actually* live in?
- 100 features might lie on a 5-dimensional manifold
- Methods: scree plot (variance threshold), correlation dimension, MLE estimators

Practical heuristic: Choose k components that explain $\geq 95\%$ cumulative variance. This gives a good estimate of intrinsic dimensionality.

EXERCISE 4.3: PCA, T-SNE & UMAP

Task: Apply PCA to e-commerce data. Compare t-SNE vs UMAP.

- Compute PCA, interpret first 3 components via loadings
- Plot scree curve; choose k for 95% variance
- Compare t-SNE (perplexity 5, 30, 50) vs UMAP (n_neighbours 5, 15, 50)
- Demonstrate reconstruction error trade-off

4.4 ANOMALY DETECTION

Outlier discovery — finding what doesn't belong

STATISTICAL OUTLIER DETECTION

Z-SCORE METHOD

$$z = \frac{x - \bar{x}}{s}$$

Outlier if $|z| > 3$ (3-sigma rule)

IQR METHOD

Outlier if $x < Q_1 - 1.5 \cdot IQR$ or $x > Q_3 + 1.5 \cdot IQR$

Robust to non-normal distributions

Z-score assumes normality. IQR is more robust for skewed data. **Winsorisation:** cap outliers at a percentile instead of removing them.

ISOLATION FOREST

- Build random trees that isolate points by random splits
- Anomalies are isolated in **fewer splits** (shorter path length)
- Normal points require many splits to isolate

$$s(x, n) = 2^{-\frac{E[h(x)]}{c(n)}}$$

$h(x)$ = path length, $c(n)$ = avg path length in a BST of n points. Score near 1 = anomaly.

Why it works: anomalies are few and different. Few = isolated quickly by random splits. Different = separated early in the tree.

LOCAL OUTLIER FACTOR (LOF)

$$\text{LOF}_k(x) = \frac{\sum_{o \in N_k(x)} \frac{\text{lrd}_k(o)}{\text{lrd}_k(x)}}{|N_k(x)|}$$

Ratio of local reachability densities. $\text{LOF} \gg 1$ = anomaly (much less dense than neighbours)

- Density-based: compares a point's density to its neighbours'
- Detects anomalies in **varying-density** data where Isolation Forest may struggle
- More expensive: $O(n^2)$ for naive implementation

SCORE BLENDING: COMBINING DETECTORS

- No single detector is best everywhere
- Normalise scores to $[0, 1]$ across detectors
- Combine: simple average, weighted average, or max-vote
- Blended scores are more robust than any individual method

Pattern: Z-score catches distributional outliers. Isolation Forest catches multivariate anomalies. LOF catches density anomalies. Together: comprehensive coverage.

KAILASH BRIDGE: ENSEMBLEENGINE FOR ANOMALY DETECTION

THEORY → ENGINE

Score blending, stacking, bagging, boosting → EnsembleEngine

```
from kailash_ml import EnsembleEngine

ensemble = EnsembleEngine()
ensemble.add("iforest", IsolationForest(contamination=0.05))
ensemble.add("lof", LocalOutlierFactor(n_neighbors=20))
ensemble.add("zscore", ZScoreDetector(threshold=3.0))

# Blend scores with automatic normalisation
blended = ensemble.blend(df, method="weighted_average")
anomalies = blended[blended["anomaly_score"] > 0.8]
```

ANOMALY DETECTION IN PRODUCTION

USE CASES

- Fraud detection (financial transactions)
- Network intrusion detection
- Manufacturing quality control
- Data drift monitoring (M3.8)

PRODUCTION CONSIDERATIONS

- Contamination rate: how many anomalies expected?
- False positive cost vs false negative cost
- Online vs batch detection
- Explainability: WHY is this an anomaly?

EXERCISE 4.4: FINANCIAL ANOMALY DETECTION

Task: Detect anomalies in financial transaction data.

- Apply Z-score, Isolation Forest, and LOF separately
- Blend scores using EnsembleEngine
- Compare: which transactions are flagged by all three vs only one?
- Business interpretation: true anomalies vs false positives?

4.5 MARKET BASKET ANALYSIS

Co-occurrence pattern discovery — finding what appears together

ASSOCIATION RULES: THE METRICS

Support: $\text{supp}(X) = \frac{\text{count}(X)}{\text{total transactions}}$

Confidence: $\text{conf}(X \rightarrow Y) = \frac{\text{supp}(X \cup Y)}{\text{supp}(X)}$

Lift: $\text{lift}(X \rightarrow Y) = \frac{\text{conf}(X \rightarrow Y)}{\text{supp}(Y)}$

Lift > 1 = positive association. Lift = 1 = independent. Lift < 1 = negative association.

APRIORI VS FP-GROWTH

	Apriori	FP-Growth
Strategy	Generate candidates, prune by min support	Compress into FP-tree, mine directly
Passes	Multiple passes over data	Two passes (build tree, mine)
Speed	Slow for large itemsets	Much faster — no candidate generation
Memory	Many candidates in memory	Compact tree representation

In practice: FP-Growth is preferred for large datasets. Apriori is pedagogically simpler.

FROM RULES TO FEATURES

Association rules are not a dead end. They connect forward in two ways:

1. **Rules as features:** "bought_bread_AND_butter" becomes a binary feature for a supervised model (M3)
2. **Co-occurrence → embeddings:** "finding patterns in co-occurrence data" is exactly what collaborative filtering does with latent factors (4.7)

APPLICATIONS BEYOND RETAIL

- **Web analytics:** page click sequences
- **Medicine:** co-diagnoses, drug interactions
- **Cybersecurity:** attack pattern signatures
- **Bioinformatics:** gene co-expression
- **Telecommunications:** service bundle optimisation
- **Education:** course co-enrolment patterns

EXERCISE 4.5: MARKET BASKET ANALYSIS

Task: Analyse Singapore retail transaction data.

- Implement Apriori and FP-Growth; find top association rules
- Compute support, confidence, and lift for each rule
- Interpret business meaning: which rules are actionable?
- Create binary features from top rules; test if they improve a classification model

4.6 NLP — TEXT TO TOPICS

Text feature discovery — extracting meaning from unstructured text

TEXT AS DATA: REPRESENTATION

- ML models need numbers — text must be converted to vectors
- **Bag of Words:** count word occurrences (ignores order)
- **TF-IDF:** weight words by importance (frequent locally, rare globally)
- **Embeddings:** dense vectors capturing semantic meaning (4.6+)

Progression: BoW → TF-IDF → Word2Vec → Transformers. Each step captures more meaning with less manual effort.

TF-IDF: DERIVATION

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \log \frac{N}{\text{df}(t)}$$

tf = term frequency in document d , df = number of documents containing t , N = total documents

WHY IT WORKS

- "the" appears everywhere → low IDF → low weight
- "kailash" appears in few docs → high IDF → high weight

BM25 EXTENSION

- Adds term frequency saturation
- Normalises by document length
- Standard in search engines

WORD EMBEDDINGS (TOOLS, NOT DERIVATION)

Method	Idea	Strength
Word2Vec (CBOW)	Predict word from context	Fast, good for syntax
Word2Vec (Skip-gram)	Predict context from word	Better for rare words
GloVe	Global co-occurrence statistics	Captures global patterns
FastText	Subword embeddings	Handles out-of-vocabulary words

Core idea: "Words that appear in similar contexts have similar meanings." The resulting vectors capture semantic relationships: king – man + woman $\approx$ queen.

How does Word2Vec learn these vectors? Through a neural network — we see this in 4.8 when we study how hidden layers create embeddings.

LDA: LATENT DIRICHLET ALLOCATION

$$P(\text{word} \mid \text{doc}) = \sum_{k=1}^K P(\text{word} \mid \text{topic}_k) \cdot P(\text{topic}_k \mid \text{doc})$$

Each document = mixture of topics. Each topic = distribution over words.

- Generative model: assumes documents were produced by sampling topics, then words
- Inference: reverse the process to discover topics
- Output: per-document topic proportions — these are **features**

NMF & BERTOPIC

NMF

- Factor TF-IDF matrix: $V \approx WH$
- W = document-topic, H = topic-word
- Non-negative constraint → additive, interpretable parts

BERTOPIC

- Transformer embeddings + UMAP + HDBSCAN + c-TF-IDF
- Captures semantic meaning, not just word counts
- State of the art for topic modelling

Method pipeline: BERTopic uses UMAP (4.3) for dimensionality reduction and HDBSCAN (4.1) for clustering. Everything connects.

TOPIC COHERENCE: EVALUATING QUALITY

$$\text{NPMI}(w_i, w_j) = \frac{\log \frac{P(w_i, w_j)}{P(w_i) \cdot P(w_j)}}{-\log P(w_i, w_j)}$$

Normalised Pointwise Mutual Information. Range: $[-1, 1]$. Higher = more coherent topic.

- **NPMI:** do topic words co-occur more than chance? (external corpus)
- **UMass:** do topic words co-occur in the training corpus?
- Complement with human evaluation: do topic labels make sense?

SENTIMENT ANALYSIS: TEXT CLASSIFICATION

- Application of text classification (supervised, using text features from this lesson)
- Pipeline: text → TF-IDF or embeddings → classifier (M3)
- Common targets: positive/negative/neutral, 1-5 star ratings

USML connection: Topic proportions from LDA/BERTopic become *features* for sentiment classification. Unsupervised topic discovery feeds supervised prediction.

EXERCISE 4.6: TOPIC MODELLING & SENTIMENT

Task: Extract topics from Singapore news articles.

- Apply TF-IDF + NMF, LDA, and BERTopic
- Compare topic quality using NPMI coherence
- Assign human-readable labels to discovered topics
- Classify customer review sentiment using topic features

4.7 RECOMMENDER SYSTEMS

THE PIVOT — optimisation drives feature discovery

CONTENT-BASED VS COLLABORATIVE FILTERING

CONTENT-BASED

- Recommend items similar to what user liked
- Uses item features (genre, price, description)
- No cold-start for items (if features exist)

COLLABORATIVE FILTERING

- Recommend items that similar *users* liked
- Uses only interaction data (ratings, clicks)
- Discovers latent preferences

USER-BASED VS ITEM-BASED CF

	User-Based CF	Item-Based CF
Find similar...	Users	Items
Recommend...	What similar users liked	Items similar to what user liked
Cold start	Hard for new users	Hard for new items
Stability	Changes as user base grows	More stable over time
Scale	Expensive with many users	Expensive with many items

MATRIX FACTORISATION: THE CORE IDEA

$$\min_{U,V} \sum_{(u,i) \in \text{observed}} (r_{ui} - \mathbf{u}_u^T \mathbf{v}_i)^2 + \lambda(\|\mathbf{u}_u\|^2 + \|\mathbf{v}_i\|^2)$$

Factorise $R \approx UV^T$. U = user embeddings, V = item embeddings. λ = regularisation.

- Each user = a vector of k latent preferences
- Each item = a vector of k latent attributes
- Rating = dot product of user and item vectors

Connection to PCA (4.3): PCA factorises $X = U\Sigma V^T$. Collaborative filtering factorises $R = UV^T$. Same algebra, different data.

ALS: ALTERNATING LEAST SQUARES

ALTERNATING OPTIMISATION

1. Fix V , solve for U : $\mathbf{U} = (\mathbf{V}^T \mathbf{V} + \lambda \mathbf{I})^{-1} \mathbf{V}^T \mathbf{R}^T$
2. Fix U , solve for V : $\mathbf{V} = (\mathbf{U}^T \mathbf{U} + \lambda \mathbf{I})^{-1} \mathbf{U}^T \mathbf{R}$
3. Repeat until convergence

Why ALS? The joint problem $\min_{U,V}$ is non-convex. But fixing one and solving the other is a least-squares problem (convex, closed-form). Alternating makes progress each step.

IMPLICIT FEEDBACK & HYBRID SYSTEMS

EXPLICIT VS IMPLICIT

- **Explicit:** ratings (1-5 stars)
- **Implicit:** clicks, views, purchases, time spent
- Implicit is more abundant but noisier
- SVD++ extends SVD with implicit signals

HYBRID SYSTEMS

- Combine content-based + collaborative
- Content features solve cold-start
- CF captures latent preferences
- Most production systems are hybrid

THE PIVOT: OPTIMISATION DRIVES FEATURE DISCOVERY

Matrix factorisation learns user and item *embeddings* by minimising reconstruction error.

This is the first time you have seen **OPTIMISATION DRIVE FEATURE DISCOVERY**.

4.7: Matrix Factorisation

$$R = UV^T$$

Embeddings via minimisation



4.8: Neural Networks

$$a = f(Wx + b)$$

Hidden activations ARE embeddings

VISUALISING LEARNED EMBEDDINGS

- Project user/item embeddings to 2D using UMAP (4.3)
- Similar items cluster together in embedding space
- Similar users cluster together in embedding space
- The latent dimensions often correspond to interpretable concepts (genre, price range, quality)

Key insight: The embeddings were not designed — they were *discovered* by optimisation. This is exactly what hidden layers do in 4.8.

EXERCISE 4.7: RECOMMENDER SYSTEMS

Task: Build recommender system on Singapore retail data.

- Implement user-based CF, item-based CF, and matrix factorisation (ALS)
- Compare recommendation quality across approaches
- Visualise learned embeddings using UMAP
- Articulate THE PIVOT: how does optimisation discover features?

4.8 DL FOUNDATIONS

Neural Networks, Backpropagation & the Training Toolkit

THE BRIDGE: FROM MATRIX FACTORISATION TO NEURAL NETWORKS



Hidden layers = automated feature engineering WITH gradient-based error feedback.

NEURAL NETWORK ARCHITECTURE



- Each node connected to all nodes in previous layer (fully connected)
- Connection has a **weight**; node has a **bias**
- Node value = weighted sum of inputs + bias, then activation function

FORWARD PASS

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b} \quad \mathbf{a} = f(\mathbf{z})$$

$\mathbf{W}$ = weight matrix, $\mathbf{b}$ = bias vector, f = activation function, $\mathbf{a}$ = activations (output of this layer)

EXAMPLE: ONE HIDDEN LAYER

Input $\mathbf{x} \rightarrow \mathbf{z}_1 = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1 \rightarrow \mathbf{a}_1 = f(\mathbf{z}_1) \rightarrow \mathbf{z}_2 = \mathbf{W}_2\mathbf{a}_1 + \mathbf{b}_2 \rightarrow \hat{y} = \mathbf{z}_2$

LINEAR REGRESSION AS A NEURAL NETWORK

- Zero hidden layers: $\hat{y} = \mathbf{w}^T \mathbf{x} + b$ — this IS linear regression (M3)
- Add one hidden layer with activation $\rightarrow$ model learns nonlinear features
- Add more hidden layers $\rightarrow$ model "writes its own parametric function"

Universal approximation theorem: A network with 2+ hidden layers and nonlinear activations can represent ANY continuous function (given enough neurons).

LOSS FUNCTIONS & GRADIENT DESCENT

$$\text{SSE} = \frac{1}{2} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad \frac{\partial J}{\partial w} = -x \cdot (y - \hat{y})$$

GRADIENT DESCENT UPDATE

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{\partial J}{\partial w}$$

η = learning rate. Iterate until loss converges.

Intuition: The gradient points "uphill." Subtracting it moves "downhill" towards lower loss.

BACKPROPAGATION: CHAIN RULE THROUGH LAYERS

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$

Chain rule: propagate error gradients backward through each layer

- Forward pass: compute predictions
- Compute loss: predicted vs actual
- Backward pass: compute gradient of loss w.r.t. each weight
- Update: adjust all weights in the direction that reduces loss

HIDDEN LAYERS = AUTOMATED FEATURE ENGINEERING

Representation learning: hidden layer activations encode learned knowledge.

These activations ARE embeddings — just like the matrix factorisation embeddings from 4.7.

"Unsupervised meets supervised": Hidden layers perform unsupervised feature discovery (like PCA, clustering). The output layer performs supervised prediction (like M3). Backpropagation couples them with error feedback.

ACTIVATION FUNCTIONS

Function	Formula	Use	Notes
ReLU	$\max(0, z)$	Hidden layers (default)	Can "die" if $z < 0$ always
Leaky ReLU	$\max(0.01z, z)$	Hidden layers	Prevents dying ReLU
GELU	$z \cdot \Phi(z)$	Transformers	Smooth approximation of ReLU
Sigmoid	$\frac{1}{1+e^{-z}}$	Binary output	Saturates $\rightarrow$ vanishing gradients
Tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	RNNs	Zero-centred but still saturates
Softmax	$\frac{e^{z_i}}{\sum_j e^{z_j}}$	Multi-class output	Outputs sum to 1

DROPOUT & BATCH NORMALISATION

DROPOUT

- Randomly zero out neurons during training
- Rate: typically 0.1–0.5
- Prevents co-adaptation
- Off during inference

BATCH NORMALISATION

$$y = \gamma \frac{x - \mu_B}{\sigma_B^2 + \epsilon} + \beta$$

- Normalise layer inputs to $\mu = 0, \sigma = 1$
- Stabilises training, enables higher LR
- Layer norm for transformers (M5)

WEIGHT INITIALISATION

Method	Scale	Best For
Xavier / Glorot	$\frac{2}{n_{\text{in}} + n_{\text{out}}}$	Sigmoid, Tanh
Kaiming / He	$\frac{2}{n_{\text{in}}}$	ReLU family

Why not zero init? All neurons compute the same gradient → symmetry is never broken → all neurons learn the same thing. Random init breaks symmetry.

OPTIMISERS: FROM SGD TO ADAM

Optimiser	Idea	Pros	Cons
SGD	Raw gradient	Simple, good generalisation	Slow, sensitive to LR
SGD + Momentum	Exponential moving avg of gradients	Faster convergence	Extra hyperparameter
RMSProp	Adaptive per-param LR (second moment)	Good for non-stationary	Can diverge
Adam	Momentum + RMSProp	Fast, robust default	May generalise worse
AdamW	Adam + decoupled weight decay	Best default for DL	Slightly more complex

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Adam: first moment (momentum) and second moment (adaptive LR)

LOSS FUNCTIONS TAXONOMY

Loss	Task	Formula
MSE	Regression	$\frac{1}{n} \sum (y - \hat{y})^2$
MAE	Robust regression	$\frac{1}{n} \sum y - \hat{y} $
Cross-Entropy	Classification	$-\sum y_k \log \hat{y}_k$
Binary CE	Binary classification	$-[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$
Focal Loss	Imbalanced classes	$-(1 - \hat{y})^\gamma y \log \hat{y}$
Contrastive	Similarity learning	Pull similar, push dissimilar
KL Divergence	Distribution matching	$\sum P(x) \log \frac{P(x)}{Q(x)}$
Reconstruction	Autoencoders	$\ x - \hat{x}\ ^2$

LEARNING RATE SCHEDULES & EARLY STOPPING

LR SCHEDULES

- **Step decay:** halve LR every N epochs
- **Cosine annealing:** smooth decay to 0
- **Warmup + cosine:** increase then decay
- **One-cycle:** increase then decrease
- **ReduceLROnPlateau:** reduce when val loss stalls

GRADIENT CLIPPING & EARLY STOPPING

- **Gradient clipping:** cap gradient magnitude to prevent explosion
- **Early stopping:** stop when validation loss increases for p consecutive epochs (patience)
- Both prevent wasted compute and overfitting

REGRESSION VS CLASSIFICATION OUTPUT

Task	Output Nodes	Activation	Loss
Regression	1	None (linear)	MSE / MAE
Binary Classification	1	Sigmoid	Binary CE
Multi-class (C classes)	C	Softmax	Cross-Entropy

Everything else is the same. Same hidden layers, same backprop, same optimiser. Only the output layer and loss change.

KAILASH BRIDGE: ONNXBRIDGE FOR NEURAL NETWORKS

THEORY → ENGINE

Neural network training, export, inference → OnnxBridge

```
from kailash_ml import OnnxBridge, TrainingPipeline

# Train with Kailash
pipeline = TrainingPipeline(
    model="neural_network",
    hidden_layers=[128, 64, 32],
    activation="relu",
    optimizer="adam",
    epochs=100,
    early_stopping=True
)
pipeline.fit(X_train, y_train)

# Export to ONNX for cross-platform inference
bridge = OnnxBridge()
bridge.export(pipeline.model, "model.onnx")

# Load and run anywhere
loaded = bridge.load("model.onnx")
predictions = loaded.predict(X_test)
```

EXERCISE 4.8: NEURAL NETWORK FROM SCRATCH

Task: Build a 3-layer neural network for HDB price prediction.

- Implement forward pass, loss (SSE), backpropagation, gradient descent
- Add dropout, batch norm, Adam optimiser, LR schedule
- Compare training curves with/without each technique
- Export trained model with OnnxBridge
- Articulate: "hidden layers are USML with error feedback"

MODULE 4: THE COMPLETE ARC

Manual (M1–M3)
 $n \rightarrow n$ features



USML (4.1–4.7)
 $n \rightarrow 1$ or k features



DL (4.8 & M5)
 $n \rightarrow m$ features

Lesson	Method	Feature Output	Kailash Engine
4.1	Clustering	Cluster labels ($n \rightarrow 1$)	AutoMLEngine
4.2	EM / GMM	Soft probabilities ($n \rightarrow k$)	AutoMLEngine
4.3	PCA / t-SNE / UMAP	Components ($n \rightarrow k$)	ModelVisualizer
4.4	Anomaly Detection	Anomaly scores ($n \rightarrow 1$)	EnsembleEngine
4.5	Association Rules	Rule features ($n \rightarrow$ binary)	—
4.6	NLP / Topics	Topic proportions ($n \rightarrow k$)	AutoMLEngine
4.7	Recommenders	Embeddings ($n \rightarrow k$) — THE PIVOT	—
4.8	Neural Networks	Hidden activations ($n \rightarrow m$)	OnnxBridge

MODULE 4 ASSESSMENT

QUIZ

- Clustering algorithm selection
- EM convergence properties
- PCA vs t-SNE vs UMAP
- Anomaly detection methods
- Association rule metrics
- Topic coherence evaluation
- Matrix factorisation & the pivot
- Neural network training toolkit

PROJECT

End-to-end pipeline:

1. Cluster data (4.1)
2. Reduce dimensions (4.3)
3. Detect anomalies (4.4)
4. Build neural network on discovered features (4.8)
5. Compare: does USML preprocessing improve DL?

LOOKING AHEAD: MODULE 5

M5: LLMs, AI Agents & RAG Systems

- The neural networks from 4.8 scaled to billions of parameters
- Transformers: attention mechanism, positional encoding
- Large Language Models: GPT architecture, fine-tuning
- AI Agents: autonomous systems that use LLMs as reasoning engines
- RAG: retrieval-augmented generation for grounded responses

The thread continues: 4.8's hidden layers → M5's transformer layers → attention IS a soft clustering of tokens → the spectrum never ends.

MODULE 4 COMPLETE

UNSUPERVISED ML & ADVANCED TECHNIQUES FOR INSIGHTS

MLFP — ML Foundations for Professionals — Terrene Open Academy

"The features that matter most are the ones no human thought to create."